

Universitat Politècnica de Catalunya

Semantic Data Management

Graph Representation Learning Lab

Master in Data Science

May 22, 2026

Authors: Laia Jané, Runxiao Qiu
Course: 2025–2026

Contents

A	Getting Familiar with KGEs	i
A.1	The most basic model	i
A.2	Improving TransE	i
A.3	Training KGEs	i
A.4	Negative Sampling	ii
A.5	Disconnected Paper	ii
B	Exploiting the Global Graph Structure	iii
B.1	Let's forget about the graph	iii
B.2	Exploiting the graph structure	iii
B.3	Tricking the GNN	iv
B.4	The more the merrier?	iv
B.5	What if we do not have initial information?	v
	B.5.1 Random Initialization	v
	B.5.2 KGE Initialization	v
B.6	Link Prediction with GNNs	v

List of Figures

A Getting Familiar with KGEs

The PubMed citation dataset was transformed into a simple knowledge graph where each paper is an entity and each citation is represented with the single relation `cites`. The graph was split into training, validation and test triples with PyKEEN and used to train and evaluate knowledge graph embedding models.

A.1 The most basic model

We first trained a TransE model as a simple and interpretable baseline. TransE represents each triple by the approximation

$$h + r \approx t,$$

so for a citation triple `paper_i cites paper_j`, the embedding of the citing paper plus the relation vector should be close to the embedding of the cited paper.

To use this geometry, we selected a paper with many outgoing citation edges, `paper_11450`. We averaged the embeddings of the papers it cites and subtracted the learned `cites` relation vector. This produced a query vector for a paper expected to cite works similar to those cited by `paper_11450`. The closest different paper to this vector was `paper_11449`. The selected paper itself was also close to the query, which is a useful sanity check because the query was built from its own citation neighborhood.

The 2D sketch in the notebook projects the chosen paper, several cited papers, the mean cited-paper vector, the query vector and the retrieved paper using PCA. The plot illustrates the TransE reasoning: adding the citation relation vector to a citing paper should move it toward the region of the papers it cites.

A.2 Improving TransE

TransE is easy to interpret, but it has an important limitation: it represents each relation as a single translation vector. This is restrictive for one-to-many, many-to-one and many-to-many relations.

In PubMed, the relation `cites` is not one-to-one. One paper can cite many different papers, and one paper can also be cited by many different papers. In a one-to-many case, TransE must satisfy

$$h + r \approx t_1, \quad h + r \approx t_2, \quad h + r \approx t_3,$$

for several different tails. Since the left-hand side is the same, TransE is pushed to place the different tail entities close together even when they are distinct papers. Similarly, in a many-to-one pattern, different heads may be forced to become too similar because they cite the same paper.

A model that alleviates this issue is TransH. Instead of applying the same translation directly in the entity space, TransH projects entity embeddings onto a relation-specific hyperplane and performs the translation there. This gives the model more flexibility and reduces the embedding collapse that can appear in TransE for one-to-many and many-to-one relations.

A.3 Training KGEs

We compared four KGE models: TransH and RotatE as translational models, and DistMult and ComplEx as semantic matching models. For each model, we explored the effect of embedding dimension, number of negative samples per positive sample and loss margin. We then selected the best configuration per model according to MRR and retrained each model for a final comparison.

Table 1: Final tuned KGE comparison.

Model	Family	Dim.	Neg.	Hits@1	Hits@3	MRR
RotatE	Translational	64	10	0.634	0.733	0.696
DistMult	Semantic	128	1	0.327	0.453	0.419
TransH	Translational	64	10	0.002	0.009	0.014
ComplEx	Semantic	64	10	0.001	0.003	0.006

The best model was RotatE, with MRR 0.695565, Hits@1 0.634010 and Hits@3 0.732882. This means that the correct entity was often ranked very close to the top. DistMult was the second-best model, while TransH and ComplEx performed poorly with our configuration.

Increasing the embedding dimension generally helped the stronger models because it increased their capacity. RotatE and DistMult benefited most from larger embeddings. Increasing the number of negative samples also helped RotatE considerably, which indicates that the model learned better rankings when exposed to more corrupted triples. The best loss margin depended on the model; for RotatE, margin 1.0 was better than the tested alternatives. Since RotatE achieved the best ranking metrics, its embeddings were kept as the best KGE embeddings for later parts of the lab.

A.4 Negative Sampling

We computed the corruption probabilities for uniform and Bernoulli negative sampling. Uniform sampling assigns the same probability to corrupting the head or the tail:

$$P(\text{corrupt head}) = 0.5, \quad P(\text{corrupt tail}) = 0.5.$$

For Bernoulli sampling, the probabilities depend on the average number of tails per head and heads per tail. In the full KG, the probabilities were exactly balanced because the graph was symmetrized. In the training split, they were still close to uniform.

Table 2: Negative sampling corruption probabilities.

Split	TPH	HPT	$P(\text{corrupt head})$	$P(\text{corrupt tail})$
Full KG	4.496	4.496	0.500	0.500
Training split	3.597	4.087	0.468	0.532

The result is close to uniform because PubMed has only one broad relation type, `cites`, and the graph is many-to-many. Bernoulli sampling is most useful when relations have a clear one-to-many or many-to-one structure. In this KG, we do not benefit much from Bernoulli sampling.

A.5 Disconnected Paper

If a paper neither cites any paper nor is cited by any paper, it does not appear in any training triple. Therefore, its embedding receives no meaningful gradient updates during KGE training. Its vector remains close to its random initialization and does not encode citation behavior or graph position.

If visualized in 2D with PCA or another dimensionality reduction technique, this isolated paper would appear in an arbitrary position. It may be separated from meaningful clusters because its location mostly reflects initialization noise rather than learned graph structure.

B Exploiting the Global Graph Structure

B.1 Let’s forget about the graph

As a non-graph baseline, we ignored the citation edges and trained classical classifiers using only the 500-dimensional TF-IDF feature vectors of the papers. The best untuned model was Random Forest with test accuracy 0.746.

Table 3: Feature-only baseline results.

Model	Test accuracy
Random Forest	0.746
Linear SVM	0.735
SVM (Sigmoid Kernel)	0.733
Logistic Regression	0.729
SVM (RBF Kernel)	0.707
SVM (Linear Kernel)	0.642
SVM (Polynomial Kernel)	0.443

The TF-IDF features already contain useful semantic information about the papers. Linear SVM, sigmoid SVM and Logistic Regression were close to Random Forest, which is expected for sparse text-like features. The polynomial kernel performed poorly, likely because the training set is small and the decision boundary becomes harder to estimate reliably.

We also tuned the strongest feature-only models. Tuning improved Random Forest only slightly, from 0.746 to 0.749.

Table 4: Tuned feature-only results.

Model	Best CV accuracy	Test accuracy
Random Forest	0.817	0.749
Logistic Regression	0.700	0.656
Linear SVM	0.700	0.656
SVM (RBF Kernel)	0.550	0.622

B.2 Exploiting the graph structure

We then trained GNN architectures that combine node features with citation structure. All models used two GNN blocks and a final linear classifier, with a 256-dimensional representation before the classification layer.

The tested architectures were GCN, GAT and GraphSAGE. GCN performs normalized neighborhood aggregation, GraphSAGE combines each node with an aggregated neighborhood representation, and GAT uses attention weights so that not all neighbors contribute equally.

Table 5: Untuned GNN architecture comparison.

Architecture	Test accuracy
GAT	0.772
GCN	0.757
GraphSAGE	0.755

All three GNNs outperformed the tuned feature-only Random Forest baseline. This shows that the citation graph provides additional signal for the semi-supervised node classification task. The best untuned GNN was GAT with accuracy 0.772, suggesting that attention over citation neighbors is useful in this dataset.

We also tuned the GNN hyperparameters over hidden size, learning rate and weight decay. The best configuration was GAT with 256 hidden channels, learning rate 0.01, weight decay 0.001 and 500 epochs. Retraining this configuration gave a final test accuracy of 0.782.

Table 6: Final comparison of feature-only and graph-based models.

Group	Model	Test accuracy
Feature-only	Random Forest	0.746
Tuned feature-only	Random Forest	0.749
Untuned GNN	GAT	0.772
Tuned GNN	GAT	0.782

The tuned GAT improves by 0.033 over the tuned feature-only baseline. This supports the main idea from the course materials: GNNs are useful because message passing combines node attributes with neighborhood information, allowing label-relevant information to propagate through the graph.

B.3 Tricking the GNN

We investigated how to manipulate the trained GNN by adding three fake citation edges to make a Type 1 paper be classified as Type 2. The strategy was to choose a Type 1 paper that was correctly classified but close to the Type 1/Type 2 decision boundary, then connect it to three highly confident Type 2 nodes.

The selected target was node 19516. Before the attack, the model classified it correctly as class 1, but it was uncertain: the Type 1 probability was 0.499108 and the Type 2 probability was 0.489959. We then added fake links to nodes 13, 11 and 3. After adding these edges, the prediction changed to class 2.

Table 7: B.3 adversarial edge attack result.

Setting	Prediction	$P(\text{Type 1})$	$P(\text{Type 2})$
Before attack	1	0.499	0.490
After attack	2	0.005	0.994

The attack was successful without modifying the TF-IDF features of the target paper. This confirms that GNNs can be vulnerable to structural perturbations: because they aggregate messages from neighboring nodes, fake citation edges can move a node representation toward the wrong class region.

B.4 The more the merrier?

We next varied the number of GNN layers while keeping the same basic GCN design. The goal was to test whether deeper message passing improves performance.

Table 8: Effect of GNN depth.

Number of GNN layers	Test accuracy
2	0.755
4	0.760
8	0.728
16	0.329

The best result was obtained with 4 layers, with accuracy 0.760, only slightly above the 2-layer model. However, performance decreased with 8 layers and collapsed with 16 layers. This shows that more layers are not automatically better.

The main explanation is over-smoothing. Each GNN layer mixes a node representation with its neighborhood. With a few layers this is useful, but with many layers each node aggregates information from a large part of the graph and embeddings from different classes become too similar. Very deep GNNs are also harder to optimize because repeated graph convolutions, nonlinearities and dropout can weaken the useful signal before it reaches the classifier.

B.5 What if we do not have initial information?

B.5.1 Random Initialization

We replaced the original TF-IDF vectors with random 256-dimensional vectors and trained the same best-performing GNN architecture from B.2. The random initialization test accuracy was 0.339.

This is much lower than the tuned GAT with TF-IDF features, which reached 0.781 in this comparison. The result is almost the same as random guessing for a three-class problem. Therefore, the graph structure alone contains some signal, but it is not enough to solve the task well. The semantic information in the abstracts is essential for this classification problem.

B.5.2 KGE Initialization

In this setting, the initial node features are replaced with the best KGE embeddings obtained in Part A. Since PyKEEN stores entity embeddings according to its own entity identifiers, the embeddings must first be reordered so row i corresponds to `paper_i`. The same tuned GNN architecture is then trained on these KGE features.

Using the KGE embeddings as initialization, the GNN obtained a test accuracy of 0.498. This is clearly better than random initialization, which obtained 0.339, but still much lower than using the original TF-IDF features, where the tuned GNN reached 0.781.

This result makes sense. The KGE embeddings are not random: they were learned from the citation graph, so they encode structural information about where each paper is located in the citation network. This gives the GNN a more meaningful starting point than random vectors. However, the KGE embeddings do not contain the same semantic information as the TF-IDF abstract vectors. Since the classification task is about diabetes type, the textual content of each paper is highly informative. The warning about casting complex values to real appears because the best KGE model was RotatE, which represents embeddings in a complex-valued space. When these embeddings are converted to real-valued tensors for the GNN, the imaginary part is discarded, which may also reduce performance.

Table 9: B.5 initialization comparison.

Initialization	Test accuracy
Original TF-IDF features + tuned GNN	0.781
Random 256-dimensional features	0.339
KGE embeddings from A.3	0.498

Overall, the results confirm that the choice of initial node features strongly affects GNN performance. KGE initialization is useful when no real node features are available, but for PubMed classification the best option is to use the original TF-IDF features together with the GNN.

B.6 Link Prediction with GNNs

For node classification, the GNN receives the PubMed graph and outputs one diabetes class per paper. For link prediction, the task changes: instead of predicting a label for each node, we predict whether a citation edge exists between two papers.

Following the GNN recipe from the course materials, the architecture still has an input layer, GNN layers and a prediction layer. The GNN now acts as an encoder that transforms node features and the training citation graph into paper embeddings. The node-classification layer is removed and replaced with an edge decoder. This decoder receives an ordered pair of paper embeddings and outputs a score for whether the first paper cites the second.

The data split also changes. Instead of splitting labeled nodes, we split edges into train, validation and test sets. We also need negative examples, i.e. pairs of papers without a citation edge, so that the model learns to assign high scores to real citations and low scores to nonexistent links.

Because citations are directed, the decoder should be able to represent direction. A symmetric dot-product decoder is simple, but it gives the same score to both directions. A better option is an asymmetric decoder, such as an MLP decoder over the ordered pair of embeddings or a bilinear decoder with trainable parameters. The loss should be changed from node-classification cross-entropy to a binary or ranking loss over positive and negative edges. Suitable evaluation metrics include AUC, average precision, Hits@K and MRR.